'*int64*' data type is used to represent integers in this example. Thus, with this example, it is clear that a seamless conversion between data types in NumPy and TensorFlow is possible.

A lot of operations are supported by TensorFlow. These operations become more and more complex as the size of the data being processed increases. TensorFlow supports arithmetic operations like *multiply*( ) (*x * y*) which multiplies *x* and *y*, *subtract*( ) (*x - y*) which subtracts x from y, *divide*( ) (*x / y*) which divides x by y, *pow*( ) (*x ** y*) which returns x raised to the power of y, and *mod*( ) (*x % y*) which returns x modulo y. An important point to remember is that if *x* and *y* are lists or tuples, then the operations are performed element-wise. TensorFlow also supports logical, relational and bit-wise operations. Here, too, the operations are performed element-wise. Figure 2 shows a Python script in which these element-wise operations are carried out. The line of code, '*t1 = tf.constant([3, 4, 2])*' creates a tensor from the list '*[3, 4, 2]*' and stores it in a variable called *t1*. The function *constant( )* is used to create a tensor from Python objects like lists, tuples, etc. Line 2 creates another tensor and stores it in a variable called *t2*. Both the lines of code, '*print(tf.pow(t1, t2))*' and '*print(t1 ** t2)*', perform an element-wise exponentiation and print the output. From Figure 2 it is clear that the result of this exponentiation, *[9 64 64]*, is the same as *[3^2 4^3 2^6]*. The line of code '*print(t1>t2)*' compares the elements of tensors *t1* and *t2* and prints results. From Figure 2, it can be verified that the output [*True True False*] is the result of the comparisons *3>2*, *4>3*, and *2>6*, respectively. Similarly, the output of line 6 can also be understood.

Now, let us see one more example, this time dealing with matrices. Consider the program shown in Figure 3. Lines 1 and 3 construct two matrices named *x* and *y*. Lines 2 and 4 print the shapes of matrices *x* and *y*, respectively. From the output of the code shown in Figure 3, it can be seen that *x* is of shape *(3, 3)* and *y* is of shape *(3,)*. Further, from our earlier discussions in this series, we know that these two matrices cannot be multiplied as such. Hence, in line 5 we apply a nice technique by which the dimension of matrix *y* is increased by 1. In line 6, we again print the shape of matrix *y*. From the output, we can see that this time the shape of matrix *y* is *(3, 1)*. Now matrices *x* and *y* can be multiplied. Finally, in line 7 these matrices are multiplied and the output is printed. Notice that similar operations can be performed on tensors also, and TensorFlow scales well even when the dimension of the tensor goes very high. In the coming articles in this series, we will learn more about data types and other complex operations supported by TensorFlow.

```
1 import numpy as np
2 import tensorflow as tf
3 arr = np.array([11, 22, 33])
4 ten = tf.multiply(arr, 3)
5 print(type(arr))
6 print(type(ten))
7 print(ten)
```

```
<class 'numpy.ndarray'>
<class 'tensorflow.python.framework.ops.EagerTensor'>
tf.Tensor([33 66 99], shape=(3,), dtype=int64)
```

Figure 1: Our first Python script using TensorFlow

```
1 t1 = tf.constant([3, 4, 2])
2 t2 = tf.constant([2, 3, 6])
3 print(tf.pow(t1, t2))
4 print(t1 ** t2)
5 print(t1>t2)
6 print(t1<t2)
```

```
tf.Tensor([ 9 64 64], shape=(3,), dtype=int32)
tf.Tensor([ 9 64 64], shape=(3,), dtype=int32)
tf.Tensor([ True  True False], shape=(3,), dtype=bool)
tf.Tensor([False False  True], shape=(3,), dtype=bool)
```

Figure 2: Element-wise operation in TensorFlow

Since this is our first discussion on TensorFlow, I think I should also mention Keras. It is a software library that provides a Python interface for the TensorFlow library. In the later articles in this series, we will also get familiar with Keras.

Now, we need to answer the following important question: How do we make use of a non-NVIDIA GPU? Well, there are many different ways to tap the power of these GPUs also. There are many powerful packages to do so. One such package is PyOpenCL. It allows us to use OpenCL (open computing language), a framework for writing parallel processing programs within Python. OpenCL can interact with GPUs manufactured by AMD, Arm, Nvidia, etc. But there are other options also. One such option is Numba, a JIT (just-in-time) compiler that can parallelize some parts of the Python code we have